

requires computing and comparing  $D(v, A_o)$  and  $D(v, A_i)$ ,  $i \in 1, 2$  for *all vectors* in the dataset. To minimize the cost, LIRE only examines nearby postings for reassignment check by selecting several  $A_o$ 's nearest postings, over which two condition checks were applied to generate the final reassign set. Experiments in Section 5 show empirically that only a small number of nearby postings for the two necessary condition checks is enough to maintain the index quality.

After obtaining vector candidates for reassignment, LIRE executes the reassignment. For vector candidate  $v$ , LIRE first searches  $v$ 's new closest posting, then performs NPA check to get rid of false-positives: if a vector actually does not need reassignment, the reassign operation is aborted. Otherwise, LIRE appends  $v$  in the newly identified posting that is NPA-compliant and then deletes  $v$  in the original posting.

### 3.4 Split-Reassign Convergence

In this section we prove that a split-reassign action to the vector index, despite the potential of triggering cascading split-reassign actions, will converge to a final state and terminate in finite steps. We first formally define the states of vector index and the events triggering state transitions. Then we prove that state transition will converge and terminate.

**Index State:** The state of a vector index for a vector data-set  $V$  comprises of two parts.

$$C : \text{set of posting centroids.} \quad (3)$$

$$M : \text{vector membership to centroid(s).} \quad (4)$$

According to LIRE, given  $C$ , each vector in  $M$  is assigned to its nearest centroid in  $C$ , i.e.,  $M$  is *uniquely* determined by  $C$ .

**Index-State Transitions:** The state transition is triggered by two types of events:

$$E_{\text{insert}} : \text{a vector } v \text{ is inserted into the vector index.} \quad (5)$$

$$E_{\text{delete}} : \text{a vector } v \text{ is deleted into the vector index.} \quad (6)$$

A reassign of vector  $v$  is considered as an  $E_{\text{delete}}$  of  $v$  followed by an  $E_{\text{insert}}$  of  $v$ . Note that an event will change the state of  $M$ , however it would not necessarily alter the state of  $C$ .

Also note that only an event  $E_{\text{insert}}$  may incur a *split action* of the vector index, which alters the state of  $C$  and subsequently  $M$ . Since  $C$  uniquely determines  $M$ , we can only focus on the state change of  $C$ .

**Split-Reassign Convergence Proof:**  $E_{\text{delete}}$  may eventually trigger a merge during a search process (according to LIRE), and the merge obviously will terminate.

Suppose an  $E_{\text{insert}}$  triggers a sequence of changes of  $C$ , denoted as  $C_i, C_{i+1}, \dots, C_{i+N}$ . To prove the convergence is to show that  $N$  is a finite number.

We note that  $C$  has the following properties:

- $|C| \leq |V|$ : The cardinality of  $C$  is bounded and no greater than the cardinality of  $V$ , i.e., the vector dataset.

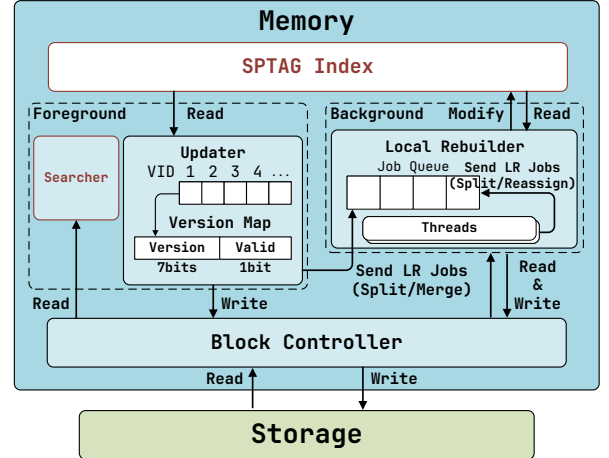


Figure 5. SPFRESH architecture (LR means Local Rebuild).

- $|C_{i+1}| = |C_i| + 1$ : Each split action will delete an old centroid from  $C_i$ , and adds two new centroids to it. Therefore, the cardinality of  $C$  *always* increases by one per split action. Based on Property 2,  $|C_{i+N}| = |C_i| + N$ . And according to Property 1,  $N \leq V - |C_i|$  because  $|C_{i+N}| \leq V$ . Since  $|V|$  is finite,  $N$  must also be finite. Therefore the split action must terminate in finite steps.  $\square$

## 4 SPFRESH Design and Implementation

### 4.1 Overall Architecture

Figure 5 shows the system architecture of SPFRESH. SPFRESH reuses the SPANN SPTAG index (depicted in Figure 3) for fast posting centroid navigation as well as its searcher to serve queries. It further introduces three new modules to implement LIRE, namely, a light-weight *In-place Updater*, a low-cost *Local Rebuilder*, and a fast storage *Block Controller*. **Updater** appends a new vector at the tail of its nearest posting and maintains a *version map* to keep track of vector deletion by setting a corresponding tombstone version to prevent deleted vectors from appearing in the search results. The map is also used to trace the replica of each vector. By increasing the version number, it marks the old replicas as deleted. The system keeps a global in-memory version map and stores vectors along with the version number on disk. A vector is stale if the in-memory version number is greater than that on the disk. This can be used for garbage collection caused by reassignment. The use of version can defer and batch the garbage collection so as to control the I/O overhead of vector removal. After vector insertions are completed in-place, the Updater checks the length of the posting and then sends a split job to *Local Rebuilder* if the length exceeds the split limit. The actual data deletions are performed asynchronously as a batch during local rebuild phase when the posting length exceeds the limit.

**Local Rebuilder** is the key component to implement LIRE. It maintains a job queue for split, merge, and reassign jobs and dispatches jobs to multiple background threads for concurrent execution.

- A *split job* is triggered by *Updater* when a posting exceeds the split limit. It cleans deleted vectors in the oversized posting and splits it into small ones if needed.
- A *merge job* is triggered by the *Searcher* if it finds some postings are smaller than a minimum length threshold. It merges nearby undersized postings into a single one.
- A *reassign job* is triggered by a split or merge job, which re-balances the assignment of vectors in the nearby postings. When the background split and merge jobs are complete, SPFRESH will update the memory SPTAG index with the new posting centroids to replace the old one.

**Block Controller** serves posting read, write, and append requests, as well as posting insertion and deletion operators on disk. It uses the raw block interface of SSD directly to avoid unnecessary read/write amplification incurred by some general storage engines, such as Log-structured-merge-tree-based KV store. Each posting may span multiple SSD blocks, each of which stores multiple vectors (including vector ID, version ID, and raw data). The *Block Controller* also maintains an in-memory mapping from the posting ID to its used SSD blocks as well as the free SSD blocks pool.

Next, we will discuss the design and implementation of *Local Rebuilder* (§4.2) and *Block Controller* (§4.3) in detail.

## 4.2 Local Rebuilder Design

In order to move split, merge, and re-assign jobs off the update critical path, SPFRESH divides the update process into two parts, a foreground *Updater* and a background *Local Rebuilder*. These two components form a feed-forward pipeline, where *Updater* is the producer of requests to the *Local Rebuilder*. In this pipeline, the background *Local Rebuilder* is a key module that implements merge, split, and reassign operators of LIRE protocol efficiently to keep up with the foreground *Updater*.

**4.2.1 Rebuild Operators of LIRE protocol.** *Local Rebuilder* implements LIRE with three basic operators.

**Merge:** To execute a merge job, the *Local Rebuilder* simply follows the merge protocol described in §3.2.

**Split:** After receiving an oversized posting split job, the *Local Rebuilder* first garbage collects deleted vectors in the posting and verifies whether the posting length after garbage collection still exceeds the split limit. If not, the *Local Rebuilder* writes the garbage-collected posting back to storage and completes the split job.

Otherwise, a *balanced clustering process* is triggered to split the oversized posting into two smaller ones. In particular, *Local Rebuilder* leverages the multi-constraint balanced clustering algorithm in [67] to generate high-quality centroids and balanced postings.

After splitting, *Local Rebuilder* puts two new postings back to the index and deletes the original oversized postings.

**Reassign:** A reassign job is generated by merge or split jobs. It checks if vectors in the new postings and/or their neighbors need to be relocated to re-balance the data distributions in the local region. The reassignment check is based on the two *necessary conditions* in §3.3. Note that neighbor posting check is *not* required for merge-triggered reassign.

Reassigning a vector without deleting its replicas in the unexamined postings increases the replica number. This not only increases storage overheads but also increases split and reassign frequency since the extraneous replicas take up spaces of postings. In order to efficiently identify stale vectors after reassignment without actual deletes, *Local Rebuilder* uses a version map to record *the version number* for each vector. A version number takes one byte and is stored in memory to record the version changes of a vector: seven bits for re-assign version and one bit for deletion label. When reassigning a vector, we increase its version number in the version map and append the raw vector data with its new version number to the target posting. All the old replicas with a stale version number are dropped during the search. The replicas will be garbage collected later.

**4.2.2 Concurrent Rebuild.** In SPFRESH, *Local Rebuilder* is multi-threaded with efficient concurrency control of updates to the in-memory and on-disk data structures. Concurrent rebuild can avoid drops of index quality due to slow re-balance.

**Concurrency Control for Append/Split/Merge:** Since append, split, and merge may update the same posting and the in-memory block mapping concurrently, We add a fine-grained posting-level write lock between these three operations to ensure a posting change is atomic.

Posting read does not require a lock. Therefore, identifying vectors for reassignment is lock-free since it only searches the index and checks the two necessary conditions. Our experiments show that even in a skewed workload, write lock contention is low, i.e., less than 1% contention cases. This is because only a small portion of postings are being edited concurrently.

During a reassign process, it is possible that a vector appends to a stale posting, which happened to be deleted concurrently. In such a case, we abort the reassignment and re-execute the reassign job for this vector. In our experiments, there are only less than 0.001% of total insertion requests encountering the posting-missing problem caused by split. As a result, the abort and re-execution overhead is minor.

**Concurrent Reassign:** SPFRESH avoids concurrently reassigning the same vector at the same time. When collecting vector candidates for reassignment, *Local Rebuilder* gathers the current version of the candidates. *Local Rebuilder* atomically executes reassignment operations by leveraging atomic primitives of compare-and-swapping (CAS) for the version